BURO

typed memory for work objects

facts, rules, and decisions about specific things you work on, available between sessions.

BURO gives an AI agent one registry of projects, services, computers, and other specific objects in your work.

Each record keeps facts about its object: purpose, locations, relationships, decisions, rules, and restrictions. The information survives between sessions and remains available whichever project the agent is working in.

BURO connects the registry to the agent and its harness through a **CLI (command-line interface)**, local or shared memory, and a reviewed way to change it. The agent or user operates BURO with terminal commands.

Either the agent or the user can maintain records. The idea is that you carry on with your work while the agent saves and updates the useful memory that comes out of it.

contents

PREPROCESSING

connection

INTRAPROCESSING

commands and records

POSTPROCESSING

memory updates

contents

Click a topic or use the PDF bookmarks.

01	what BURO remembers	3
02	install and connect	4
03	connect the agent	5
04	context at session start	6
05	commands and records	7
06	updating memory	8
07	bundled types	9
08	project and service	10
09	bundled record fields	11
10	device fields	12
11	names and permanent IDs	13
12	links between records	14
13	create a custom type	15
14	defining fields	16
15	a custom type: book	17
16	move a record to another type	18
17	keeping data safe	19
18	snapshots and recovery	20
19	local or shared memory	21
20	updates and further reading	22

what BURO remembers

BURO stores facts about specific objects. One record describes one website, book, service, computer, or other thing you work on. The agent opens the record when those facts are relevant to the task.

Term	Meaning
`object`	The thing itself: for example, a book called A History of the City.
`record`	Saved facts about that particular book: editorial requirements, decisions, and references to materials.
`record type`	The definition of its fields: what to remember about every book and what each field means.

A type defines the **format and meaning of fields**. BURO checks value formats; field guides help the agent decide what belongs there. The program does not establish whether the facts are true.

one rule is enough to start

```
id: my-site
name: My website
kind: project
important:
  - Ask me before publishing.
```

The bundled types require only id, name, and kind. The agent can prepare them. Add the rest when it is useful; a name and one rule already make worthwhile memory.

Within this scope, BURO can replace scattered AGENTS.md files, project memory files, and RAG retrieval of context about individual work objects. A prompt instruction or skill tells the agent when to use BURO. BURO itself runs no model and does not process chats in the background.

install and connect

Requires **Node.js 24.14 or newer**. Install the public release package, create memory, and connect your harness:

```
npm install -g \
  https://github.com/Krablante/buro/releases/download/v3.1.0/buro-3.1.0.tgz
buro init
buro connect opencodez
```

Init creates memory with the bundled types. For a local database it also installs a native backup schedule. Configuration, the database, and snapshots live outside the installed package.

Resource	Default location
configuration	`~/.config/buro/config.json`
database	`~/.local/share/buro/state/buro/buro.sqlite3`
snapshots	`~/.local/share/buro/state/buro/backups/sqlite/`

No config editing is needed for the ordinary local setup. Set a server URL when you want shared memory.

after connecting

Replace opencodez with opencode, codex, or claude as needed. Restart OpenCode/OpenCodez. In local Codex, review and trust the installed hook through /**hooks**, then start a new session. In Claude Code, start a new session with SessionStart enabled.

Check `buro current` and `buro help`. The first opens the initial context; the second shows commands, the connected memory, and its accepted definitions.

connect the agent

> **The agent needs instructions about BURO and terminal access to its commands.** Installing the program alone does not explain when to read and update memory.

a supplied connection

```
buro connect opencodez
```

Connect installs the **skill and a session context connection**. It also puts system-prompt.md beside SKILL.md as a ready-to-copy resource. **Connect does not edit your System prompt.** For OpenCode/OpenCodez, OPENCODE_CONFIG_DIR selects the profile when set; `--path <profile>` takes precedence.

The skill and System prompt block live together in skills/buro/. Each explains the basics and commands. You may use either or both. In OpenCodez, add the block through the System prompt library or by editing your personal prompt.

another agent

```
buro connect file --path /path/to/AGENTS.md
```

This explicit command adds instructions to the selected file. Set up automatic initial context separately when there is no supplied connection.

what the instructions explain

How to find the relevant object, read its record, and follow its rules. How to inspect accepted types and fields. How to prepare a change, review its diff, and save it through push.

Check facts when they contradict something or appear stale. Unknown facts stay unknown. Discussing an idea does not make it a decision, and a session transcript is not a substitute for useful facts. A stored record grants no permission beyond the user request.

context at session start

A connected session receives facts about its current device and associated records through **buro current**. The connection runs the command and supplies its output before the first model response. In OpenCode/OpenCodez this happens on the session first request, once per session in the running process.

The current device is where the harness runs. BURO supplies its record, when one exists, and identities of objects located there. The name comes from current_context, otherwise from the short system hostname. Having no host record is normal.

Set the list limit through current_limit in config.json; the default is 100. If more objects exist, the output reports truncation. This is part of the memory, not an inventory of every record.

> **The agent chooses the relevant object from the task and conversation.** BURO returns the requested records. No separate active-project detector is needed.

Harness	Initial context
OpenCode / OpenCodez	A plugin adds current to system context. Restart after installing the plugin.
local Codex	A trusted SessionStart hook supplies current on startup or clear.
Claude Code	A SessionStart command hook supplies current on startup or clear.

Resuming or compacting a terminal session does not run these hooks again. The agent can refresh facts with `buro current` when needed.

If BURO is unavailable, chatting still begins. The agent receives a loading error, not an empty memory. Connections ship with BURO and are refreshed by `buro update`. Codex/Claude connections target local harnesses; compatibility details are in the agent guide.

commands and records

Help explains the program and the accepted memory model. Individual record contents are opened separately.

```
buro help
```

Help shows commands, the read/edit workflow, agent connections, and memory transfer. It also shows the BURO version and memory target: a local database path or server URL.

Below that are the default type and all active types marked as bundled or custom. Each includes fields and nested fields, formats, required values, and guides. These are the definitions of the connected memory.

Command	Purpose
`buro <id-or-name>`	Open one record: rules, locations, and other visible fields.
`buro search <text>`	Find IDs, names, and aliases. Other field contents are not searched.
`buro list [kind]`	Show record IDs, names, and types.
`buro current`	Show the current device and a bounded list of associated objects.

names and result pages

Exact IDs win. An ambiguous name returns candidate IDs. For an ID matching a command name, use `buro get <id>`.

List and search accept `--limit` and `--offset`. For example:

```
buro list project --limit 100 --offset 100
```

When memory is unavailable, help still shows command guidance and reports that active definitions could not be obtained. It does not substitute bundled types. The agent does not load the entire registry when one lookup answers the task.

updating memory

By default, the agent updates memory at the end of completed work when there is a useful change, or on your direct request. You can set another write policy in your instructions.

each change gets its own file

Command	Purpose
`buro draft new <id> [kind]`	Create a record.
`buro draft pull <id>`	Edit a record.
`buro draft delete <id>`	Prepare a record deletion.

The command prints a path to a temporary YAML file. The agent or user edits it, retains the **__buro** metadata, and substitutes the printed path for `<file>`:

```
buro draft diff <file>
buro draft push <file>
```

Diff displays and validates the change. Push repeats validation of values and original revisions, takes a snapshot, then saves the change. A successful push removes only its own file. A failed push retains it. `buro draft clear <file>` deliberately discards that draft.

concurrent work

Two sessions can edit different records. If someone has changed the same record or its type definition, the stale draft is rejected. Pull a fresh draft, reconcile your intended change, and review the diff again. Type edits have the same protection.

save meaning, not a chat recap

Keep confirmed facts, decisions, and rules that will matter later. Consider the whole conversation, including recent corrections and revoked decisions. Duplicates, temporary plans, and session logs are not useful records. If there is no useful change, there is nothing to write.

bundled types

A new memory includes three ready-made types. Use the ones that fit, or keep only records of your own types. You do not need to populate every bundled type.

Type	Meaning
`project`	A product or independent undertaking: software, a website, bot, book, or research project. Optional repo links to its repository.
`service`	A technical component of the working environment: DNS, VPN, proxy, storage, or an MCP tool. Optional repo links to code or configuration.
`host`	A computer, server, or virtual machine. Alongside the common fields it has addresses, ssh_target, and git_identities.

A project can have a running installation and operational details. A service can have source code and a repository. Choose the type by its role in your work; folders and installation locations do not determine it.

locations and relationships

locations is a list of places. Each may contain host (device), path, url, and purpose. For example, website source lives on a laptop, the running installation on a server, and the public page at a URL.

related contains informational references. Its id points to a BURO record, path to a file, and url to a web resource. The note explains why the reference matters.

rename a custom type label separately

Label is the displayed type name. For a custom type, you can change it independently of the ID, fields, and records. Changing a field format is different: BURO first validates saved values.

Bundled definitions are not edited directly. Create an independent copy with `buro types new my-project --from project` when you want your own version.

project and service

A project is a product or independent undertaking. A service is a technical component of the working environment.

Services usually belong to the local working environment. Connected external tools fit too: for example, Cloudflare used as infrastructure. The program location alone does not determine its type.

Facts	project	service
purpose	What the product does, its users, and requirements for the result.	The technical capability the component provides.
source	Code, materials, build, and product structure.	Configuration, installation scripts, and service code.
running version	Where it runs; how to publish, update, and recover it.	Where the component runs; how to configure and operate it.
important rules	Product requirements, accepted decisions, and development restrictions.	Access, connection, compatibility, and operating rules.
relationships	Tools and services the project uses.	Who uses the service and which components it depends on.

A running bot can keep its code, configuration, data, and updates in one project record. A separate service record is useful for a component with an independent maintenance responsibility.

For example, the website project record describes content, build, and publishing. Caddy accepts web requests and routes them to the appropriate site or application. Its service record describes routing, certificates, and access rules. Related links connect the records; both types share the same common fields.

bundled record fields

The three bundled types share eight fields. All are optional and have the same meaning in each type:

Field	Meaning
`summary`	Text. What this object is, why it exists, and what it does.
`important`	A list. Confirmed facts, decisions, and requirements. Approval conditions belong here too.
`avoid`	A list. Other restrictions: for example, preserve a file format or keep heavy builds off this device.
`locations`	Source, running installations, data, materials, or web pages.
`related`	Informational references: a record id, file path, or resource url. The note explains the link.
`read_first`	Main documents and instructions to read when beginning relevant work.
`aliases`	Other names by which the object may be found.
`notes`	Details that need more room than the short summary.

Project and service also have optional **repo**, a string linking to a repository of code, materials, or configuration.

> **Approval conditions always belong in important.** Ask before publishing and do not publish without approval express the same rule. A negative phrasing does not move it to avoid. Do not duplicate a rule in both fields.

Id is the permanent record identifier, name is the display name, and kind is the type. All three are required; BURO manages the record revision. Your custom type may have different fields and guides.

device fields

Host describes a computer, server, or virtual machine. It has the eight common fields from the previous section and three more. **All three are optional.**

Field	Meaning
`addresses`	A list of network addresses: for example, a home-network address and a VPN address. One address or several are fine.
`ssh_target`	A name or address for SSH terminal access. For example, home-server in ssh home-server.
`git_identities`	Git/GitHub identities: commit author name and email, GitHub account, and instructions for choosing one.

For example, one git_identities entry may be for personal projects and another for work. Entries may contain user_name, user_email, gh_cli_user, and instructions. The name and email identify a commit author in Git history. A GitHub account is used to access repositories on GitHub. These are different facts.

where other facts go

Summary can say work laptop. Notes can describe the operating system and installed programs. Work requirements go in important; other restrictions in avoid. Significant device directories can be recorded in locations.

a device record is optional

Current only needs a device name. Locations.host does not require an existing host record. When a device record exists, the agent receives its facts too; otherwise the associated-object list remains available.

names and permanent IDs

Id is unique across the memory. It can be a readable name such as personal-site; numeric IDs are not required. Name is the display name, such as Website.

An ID is permanent and cannot be changed by an ordinary edit. Rename the display name and ID references still reach the same record. ID uniqueness applies across all record types.

Situation	Result
the ID is occupied	Another record cannot take it. Edit the existing record if it is the same object; choose a new ID for another object.
names overlap	Allowed. Two sites can both be called Website with IDs personal-site and company-site. Ambiguous lookup returns candidate IDs.
a new object after deletion	It may share the deleted record display name, but needs a new ID. The deleted ID remains assigned to its original object.
the original object returns	Recovery from a snapshot restores the original data and ID. Ordinary creation with a retired ID is rejected.

> Suppose other records still link to an old website as my-site. Giving that ID to a new website would silently redirect those links to another object. Retiring deleted IDs prevents this substitution.

An exact ID wins over another record name or alias. Names and aliases help you find a record; references use its exact ID.

Types follow a separate rule: **one ID, one accepted definition**. A copy receives another type ID and remains independent of its source.

links between records

locations: where an object lives

```
locations:
  - host: laptop
    path: /home/user/projects/my-site
    purpose: Source code
  - url: https://example.com
    purpose: Public website
```

A new path must be absolute. If a new path has no host, BURO adds the current client device; diff shows the addition. Locations.host contains a device name. Creating its host record is optional.

related: informational references

```
related:
  - id: voice-service
    note: Publishes article voiceovers.
  - url: https://example.com/docs
    note: Tool documentation.
```

A bundled related entry selects id, path, or url; note explains the reference. Deleting the target record does not change other records. Looking up the deleted target returns not found. Correct a reference through the ordinary diff/push workflow.

ref: a checked reference

A **ref** field points to an existing record. For example, a website main-server field points to home-server. While that reference is in use, BURO prevents deleting home-server. First remove the reference or choose another target. Ref is available in custom types and is optional.

create a custom type

BURO supplies the bundled definitions. You or the agent create and edit custom ones; BURO stores and makes them available. There are no definition directories or external paths to configure.

For example, you want a project type with only important. Start with a copy of project:

```
buro types new my-project --from project
```

The command prints a temporary YAML path. Keep the fields and guides you want, retaining __buro metadata, then:

```
buro draft diff <file>
buro draft push <file>
```

After push, **my-project is immediately available for records**. No config.json change or extra init is needed. The bundled project remains as it was; its future updates do not alter the copy.

other operations

```
buro types new <type>
buro types edit <type>
buro types delete <type>
```

Each command prepares a separate draft. Diff and push take its path, just as with a record edit. A used type can be deleted after its records are moved or explicitly deleted; checked references must remain valid too.

> **A copy is an independent definition.** Fields are not mixed in from other types or inherited behind the scenes. Field names are unique within a type; the same name can have another definition in a different type.

--from prefers a supplied bundled template when that ID exists; otherwise it copies the active custom type. A custom label can change independently of its fields.

defining fields

A type has id, optional label, and a fields object listing its own fields. A **record type** such as book and a **value format** such as string-list are different things.

Format	Value
`string / text`	Short / extended text, such as a publisher name or editorial requirements.
`boolean`	Yes or no: YAML true and false.
`integer / number`	A safe whole number / a finite number, including fractions.
`string-list`	Non-empty strings: rules, aliases, document names.
`record / record-list`	Nested fields / a list of those objects, described through fields. Empty list entries are rejected.
`ref`	An existing record ID. Optional target_kind restricts the target type.

what a field definition says

type selects the format. **guide** explains what to save. **required: true** makes a value mandatory; otherwise it is optional. Nested fields use the same definitions.

The **default** supplies an omitted value. When a type change adds defaults to older records, they are saved with the change. A later default change does not rewrite those saved values.

The **section** groups fields in record output; definition order controls their order. **packet: false** hides a field, including a nested one, from ordinary record output. It does not remove the value or protect a secret.

fields with additional behavior

String-list aliases participates in alias lookup. Standard locations associates records with devices. Both are optional. Other fields do not acquire behavior just from their names. Unknown fields and invalid formats are rejected.

a custom type: book

You work on several books. Each needs editorial requirements and publication status. **buro types new book** creates a draft in which you can define:

```
id: book
label: Book
fields:
  editorial_rules:
    type: string-list
    guide: Accepted editorial requirements for this book.
  published:
    type: boolean
    guide: Whether the book has been published.
```

After diff and push, the type is ready. **buro draft new city-book book** prepares one particular book:

```
id: city-book
name: A History of the City
kind: book
editorial_rules:
  - Preserve informal speech in quotations.
  - Agree the final text with the author.
published: false
```

This is record content; retain the generated draft __buro metadata. Review this draft with diff and apply it with push too.

> You say: let us continue editing A History of the City. The agent finds city-book, reads its requirements, and follows them. A newly accepted requirement goes into the same record.

A guide helps choose the right field. A boolean format prevents saving perhaps instead of true or false. BURO does not establish whether the book was actually published.

move a record to another type

Change a record type through ordinary draft pull, diff, and push. **The ID stays the same.** Its kind and content fields change.

example: a project becomes a book

The book used to be a project record. You have now created a custom book type. The old record has important and repo; the new one needs editorial_rules and published. You or the agent decide which fields share a meaning.

Step	Action
open a draft	Pull the current record. Inspect its values and the target type definition.
prepare the move	Change kind to book. Move important rules into editorial_rules when they are editorial requirements.
handle the rest	The populated repo has no place in the target type. Add a suitable book field, preserve the value elsewhere by agreement, or explicitly delete it.
review and save	Diff shows the type change and removed/added fields. Push validates the new record and references, then applies the complete change.

> **BURO does not guess field mappings.** Changing kind alone does not discard incompatible values. Leaving them in unknown fields rejects the push. An intentional removal is visible in the diff.

An incoming ref restricted to the former type blocks the move until corrected. Informational related links keep their meaning through the unchanged ID.

Field renames and other incompatible changes need the same care. Validate bulk migrations on a copy; import is not a shortcut for one record edit.

keeping data safe

> **An incompatible change does not break accepted memory.** BURO rejects it, explains the problem, and continues reading the previous records and definitions.

validate a type change first

The proposed definition is checked against saved records. Suppose a field contains five or six and the new format accepts one number. BURO does not pick 5 or 6. You or the agent resolve the value and retry. A partially migrated memory does not become the accepted state.

import replaces the entire memory

Export saves records, type definitions, and retired IDs. Import replaces the entire registry with them. A preview shows current/incoming counts and disappearing IDs. It does not add a few records to the existing memory.

```
buro export memory.yaml
buro import memory.yaml --dry-run
buro import memory.yaml
```

The complete input is validated. A snapshot is mandatory before replacement. A validation or snapshot error cancels the operation. Records and the model are replaced transactionally; failure rolls back the previous state.

An empty but valid import can clear the registry. The program cannot know whether the file was chosen by mistake. Recover an accidental valid replacement from a snapshot.

> A snapshot is required before changing a record, type, or whole memory. If it fails, the change is not applied. Import and recovery invalidate old drafts; prepare new ones afterwards.

Each changed write copies the complete database. I/O and storage costs grow with the memory; BURO suits durable facts rather than event streams or large binary data.

a draft is not a backup

It is a temporary file with its own path. OS temporary-directory cleanup may remove old drafts. Preserve useful unfinished edits separately. A failed push retains its file for correction.

snapshots and recovery

Three independent groups retain recent history and points from previous days:

Group	Retained
every 3 hours	Latest 8 snapshots, at least three hours apart.
daily	Latest 8 daily snapshots.
before changes	Latest 20 snapshots before record, type, or whole-memory changes. Manual snapshots use this group too.

Ordinary edits do not displace daily snapshots. An old snapshot is removed only after a successful new one in the same group.

Init on the database device installs a native schedule: a user systemd timer on Linux, LaunchAgent on macOS, or user task on Windows. Shared-memory clients install no schedule. If the scheduler is unavailable, init reports **NOT enabled** and supplies a command to schedule manually. A powered-off device makes no copies.

a manual snapshot and recovery

```
buro backup
buro restore /absolute/path/to/snapshot.sqlite3 --dry-run
buro restore /absolute/path/to/snapshot.sqlite3
```

Before recovery, stop the API and other writers. Preview the snapshot with --dry-run, restore it, read `buro list` and a known record, then resume operation. Prepare fresh drafts afterwards.

A snapshot includes records, types, retired IDs, the BURO version, and the database device context settings. Recovery restores current_context and current_limit. The target device keeps its own storage paths and transport settings.

> **If a disk fails.** Snapshots on that disk protect against an editing mistake but fail with the disk. Keep copies on another device or separate storage to protect against disk loss.

local or shared memory

Without api_url in config.json, BURO uses local SQLite. With it, BURO uses that HTTP server. **One database is enough for shared memory.** Clients read and change the same records and types.

On the database device, within a trusted private network:

```
buro serve --host 0.0.0.0 --port 8765
```

On other devices, in ~/.config/buro/config.json:

```
{
  "api_url": "http://home-server:8765",
  "current_context": "laptop",
  "current_limit": 100
}
```

Serve binds only to 127.0.0.1 by default. BURO_CONFIG selects the config file; BURO_ROOT selects the local data root. Environment settings override JSON; the full list is in the operations guide.

When the server is unavailable, BURO reports an error. It does not fall back to local storage or create another memory. A timed-out write may have completed on the server; read the affected record before retrying.

access and secrets

The built-in server has no authentication, authorization, or TLS. Every admitted client can read and change shared memory. Admit trusted clients through a restricted network, VPN, protected tunnel, or HTTPS proxy.

Keys, passwords, and tokens may be stored under the same rules as other facts. BURO does not mask values automatically: the agent sees visible fields. BURO does not encrypt the database, exports, snapshots, or ordinary HTTP. Packet: false is not access control.

updates and further reading

Run `buro update` to install the latest stable GitHub release on this device. `buro update --check` only checks. The command verifies the package, updates compatible bundled definitions in existing local memory, and refreshes agent connections. Custom types and records are preserved.

Restart connected agents and any running BURO API afterward. Older custom profiles need connect once to register their paths. Versions before 3.1 need one manual npm upgrade. Personal System prompts stay manually configured; supplied instructions are in skills/buro/.

when something fails

Situation	Next step
API unavailable	Check api_url and reachability from the device running the agent.
no initial context	Check buro current in the harness environment, the connection, and hook trust.
stale draft	Pull a fresh draft, reconcile the intended change, and review diff again.
snapshot failed	Check free space and permissions in the backup directory, then retry the retained draft.

detailed guides

- README: installation and the main entry point.
- Model: fields, references, definitions, and v2 migration.
- Agents: connections, profiles, and harness requirements.
- Operations: configuration, schedules, and recovery.
- Development: architecture, HTTP API, and package releases.

For a v2 upgrade, read the model migration procedure first and validate a memory copy. Previous definitions remain custom types under their original IDs; new templates do not replace them.

BURO validates structure and consistency of changes. You and the agent establish which facts are true, which decisions were accepted, and which rules are worth remembering.